Java Spring · Week 3

# Who hands it what it needs

wiring beans · @Autowired · scopes

Eng. Mohamed Shehab

Faculty of Computers and Information, Menoufia University

Where we stopped

# Six beans in the box, and none of them knows another

Week 2 left us here

`@Component` on the class, `@ComponentScan` on the configuration, and Spring finds it

What is still missing

Every bean stands alone. Not one of them takes anything from another

```
beans in the context: 6
    four of Spring's own
  appConfig
  bookingService
```

The hook

# A booking has to send an email. Who hands it a sender?

```
public class BookingService {
public void book(String user, String eventTitle) {
log.info(...);
???.send(user, "Booking confirmation");
}
}
```



What comes to mind first

And why that hurts

Today

# From putting a bean in the box, to wiring one to another

3.1

We wire them by hand inside the configuration

3.2

`@Autowired`, and the three places it goes

3.3

We build a loop on purpose and read Spring's answer

3.4

More than one of a type. Which one gets injected?

# Two @Bean methods, and neither knows the other

```
@Bean
public SmtpEmailSender smtpEmailSender() {
return new SmtpEmailSender();
}
@Bean
public BookingService bookingService() {
return new BookingService(); // takes nothing
}

ahmed@example.com booked a seat for Spring Boot Workshop
no sender, nothing was emailed
beans in the context: 7
service's sender: null
```

3.1.1 · Wiring by a direct call

# One @Bean method calls the other

```
@Bean
public BookingService bookingService() {
return new BookingService(smtpEmailSender());
}
```

```
BookingService created with a sender
ahmed@example.com booked a seat for Spring Boot Workshop
[SMTP] → ahmed@example.com | Booking confirmation
```



It works. But something in that output is stranger than that.

3.1.1 · The question

# We called the method ourselves. Why is there only one?

```
SmtpEmailSender created ← once, not twice

service's sender:  ...SmtpEmailSender@2e1f5c4d
context's sender:  ...SmtpEmailSender@2e1f5c4d
```

What the code says

`smtpEmailSender()` is a method call, and the method says `new`

What Spring does

It intercepts the call and hands back the bean it already has

3.1.2 · The other way

# Ask for it in the parameter and let Spring bring it

```
@Bean
public BookingService bookingService(SmtpEmailSender smtpEmailSender) {
return new BookingService(smtpEmailSender);
}
```

The output

Identical, down to the one shared address

The difference

You name what you need instead of where to get it



And that is the shape of everything else today.

3.2 · Back out of the configuration

# `@Autowired` means: fill this from the box

Until now

The configuration said: build the service, hand it a sender

From here

The class itself says: I need a sender, bring me one

The book says plainly that this is what you meet in real code, paired with `@Component`, not with `@Bean`.



So we go back to where week 2 left us: @Component and @ComponentScan.

# The same annotation, in three different places

```
@Autowired
private SmtpEmailSender emailSender; // on the field

@Autowired
public BookingService(SmtpEmailSender s) { ... } // on the constructor

@Autowired
public void setEmailSender(SmtpEmailSender s) { ... } // on the setter
```

We tried all three. And the output?

# All three print exactly the same thing

```
ahmed@example.com booked a seat for Spring Boot Workshop
[SMTP] → ahmed@example.com | Booking confirmation
beans in the context: 7
```

So the difference is not behaviour

From the console alone you cannot tell which one I used

And it is not the value either

There is one bean, so Spring puts the right one in all three



Then where is it? In one thing: what the compiler can stop you doing.

3.2 · The real difference

# With a field you can build a half-made object. With a constructor you cannot.

```
// No Spring, no context. This is what a unit test does.
BookingService s = new BookingService();
s.book("ahmed@example.com", "Spring Boot Workshop");
```

```
the compiler was happy to build this object
ahmed@example.com booked a seat for Spring Boot Workshop
Cannot invoke "EmailSender.send(...)" because "this.emailSender" is null
```



With the field

With the constructor

3.2 · And one thing the book skips

# We removed @Autowired entirely, and it still worked

```
// no @Autowired anywhere here
public BookingService(SmtpEmailSender emailSender) {
this.emailSender = emailSender;
}
```

With exactly one constructor, Spring takes it as the one you meant, without being told.



Which is why you will not see it in Spring Boot code from next week on.

# What if each one is waiting for the other?

```
public BookingService(SmtpEmailSender s) { ... }
// and
public SmtpEmailSender(BookingService b) { ... }
```

To build the first you need the second, and to build the second you need the first.

That is a deadlock. Spring has nowhere to start.

# The message draws the loop for you

```
Error creating bean with name 'bookingService':
Unsatisfied dependency ... constructor parameter 0:
Error creating bean with name 'smtpEmailSender':
Unsatisfied dependency ... constructor parameter 0:
Error creating bean with name 'bookingService':
Requested bean is currently in creation
```

Read the names in order: bookingService → smtpEmailSender → bookingService.

# We added a second sender. Which one does Spring inject?

```
No qualifying bean of type 'com.eventia.service.SmtpEmailSender'
available: expected single matching bean but found 2:
backupEmailSender,smtpEmailSender
```

A message we have seen before. Remember the two Event beans in week 2?

# Spring works down this list, in order

| # | The case | What happens |
|---|---|---|
| 1 | The parameter name matches a bean name | That bean is injected |
| 2 | No name match, and one bean is @Primary | The primary one |
| 3 | No match, no primary, but there is @Qualifier | The one it names |
| 4 | None of the above | It throws |

# Change the parameter name. Nothing else. Run it.

```
public BookingService(EmailSender backupEmailSender)
→ [BACKUP] → ahmed@example.com | ...

public BookingService(EmailSender smtpEmailSender)
→ [SMTP] → ahmed@example.com | ...
```

Not one other thing in the project changed. Only the name.

# Who decides, the one giving or the one asking?

`@Primary`

Sits on the implementation. I am the default for anyone who asks

`@Qualifier`

Sits at the point of asking. I want that one specifically

```
@Primary @Component
public class BackupEmailSender ...

// or
public BookingService(@Qualifier("smtpEmailSender") EmailSender s)
```

The interface

# The service asks for a contract, not for one implementation

```
public interface EmailSender {
void send(String to, String subject, String body);
}
@Component public class SmtpEmailSender implements EmailSender
@Component public class BackupEmailSender implements EmailSender

public BookingService(EmailSender emailSender) { ... }
```

The output

Identical. What changed is that the service no longer knows

And two implementations?

The same found 2 message, and the same three ways out

Labels

# @Service and @Repository

| Label | Goes on | Does what |
|---|---|---|
| @Component | any class | puts it in the box |
| @Service | business logic | exactly the same thing |
| @Repository | whatever talks to the database | the same, plus one job in section 7 |

Remember measuring in week 2 that @Service carries @Component inside it? This is why.

Life cycle · A misleading name

# Spring’s singleton is not the Singleton you were taught

```
// both singletons, both the same class
@Bean Printer deskPrinter() { return new Printer("desk"); }
@Bean Printer backPrinter() { return new Printer("back"); }

deskPrinter : desk@876017024
backPrinter : back@1805861882
same object? false
```

### The Singleton in design books

One instance in the whole program, and the class itself forbids a second

### The singleton in Spring

One instance per bean name, per context. The class is ordinary and new still works

# One instance for everyone means every changing field is shared

```
@Service
class InvoiceBuilder {
private int total = 0; // changes
private String customer; // and so does this
void start(String c) { customer = c; total = 0; }
void addSeat(int p) { total += p; }
}
```

```
// ahmed and mona book at the same moment
mona owes 350
mona owes 350
// expected: ahmed owes 100, and mona owes 250
```

Life cycle · The fix you will actually use

# Do not keep it in a field. Build it where you need it

```
@Service
class BillingService { // still one instance
String bill(String customer, int seatPrice, int seats) {
Invoice invoice = new Invoice(customer); // local, per call
...
return invoice.finish();
}
}

// ahmed and mona at the same moment, one service object
ahmed owes 100
mona owes 250
```



No scope, no annotation. The `Invoice` needed nothing from the container, so it was never a bean.

# `prototype`: a new one every time you ask

```
@Scope("prototype")
@Service
class InvoiceBuilder { ... }

// asked for twice
InvoiceBuilder created
InvoiceBuilder created
same object? false
```

## So when do you use it?

Java Spring · Week 3 · 25

Hardly ever. The slide before this one is what you reach for.

## Then why show it?

So you recognise it in code, and know it has catches of its own. Another time.

# @Lazy: still a singleton, but wait until someone asks

```
--- building the context ---
--- context ready, I have asked for nothing ---
TicketPrinter created ← only now
--- now I asked ---
```

| When do you reach for it? | Example |
| --- | --- |
| An expensive bean that is rarely used | a slow external system behind an admin screen |
| A bean you cannot configure locally | a payments client you do not want blocking your startup |
| A large application while you develop | shorter startup between runs |



🔴 But do not make everything lazy. You want a bad bean to fail at startup, not at two in the morning.

Next week

# So far we write all of this configuration by hand

```
var context = new AnnotationConfigApplicationContext(AppConfig.class);
// a pom with every dependency and every version typed out
// and @ComponentScan telling it where to look
```

Next week: Spring Boot. All of these lines disappear.

In short

# What we walked away with today

| | The thing |
|---|---|
| 1 | Beans are wired from the configuration, by a call or by a parameter |
| 2 | Call a @Bean method and Spring hands back the one it has |
| 3 | @Autowired in three places, and the same output from all three |
| 4 | The constructor is the one that stops you building a half-made object |
| 5 | A loop throws, and the fix is to redesign the classes |
| 6 | More than one: the name, then @Primary, then @Qualifier |
| 7 | And those same rules hold once the dependency is an interface |
| 8 | One instance for all, so keep no changing state in a field |

Java Spring · Week 3

# Questions?

Eng. Mohamed Shehab

mohamed.shehab.fci@gmail.com · Telegram @shebo280